MGSC 697 · Assignment 2 · June 2026

Retail Inventory Replenishment

Training a Q-Learning Agent to Optimise Weekly Stock Orders

By Laura Manzanos (ID: 261284603)

Core Path · Tabular Q-Learning · Custom MDP · 52-week Horizon · No external libraries

The Business Problem

Every week, one decision: how many units to order?

ORDER TOO LITTLE

Stockout penalty: $8/unit
Lost sale + lost customer
0.62% rate with Q-learning

ORDER TOO MUCH

Holding cost: $0.50/unit/week
Capital tied up in stock
Agent learns to avoid this

ORDER OPTIMALLY

Revenue: $20/unit sold
Match supply to demand
That's the RL problem

The agent acts, demand is realised, a reward arrives, and the strategy improves - exactly the RL loop from the course slides.

MDP Specification

STATE	**inventory_bin × demand_regime** 10 bins × 3 regimes = 30 states	Captures the two variables that drive the optimal order
ACTION	**Order qty ∈ {0, 5, 10, 15, 20, 25}** 6 discrete choices	Discrete set mirrors real supplier MOQ constraints
REWARD	**Revenue − holding − stockout pen. − order cost** $20 sell · $0.50 hold · $8 short · $10+$2q order	All four cost levers wired in; business strategy as a number
TRANSITION	**Demand ~ Poisson(μ_regime); regime → Markov chain** Low μ=5, Med μ=10, High μ=18	Stochastic demand + slow seasonal drift
HORIZON	**T = 52 steps (one full year)** Weekly decision cycle	Long enough for temporal credit assignment to work
DISCOUNT γ	**γ = 0.97 ≈ 33-week effective horizon** Medium-term time preference	Values this season's margin, not just next week's sale

Three Agents

Compared on the same environment with fixed seeds

RANDOM

Baseline floor

Uniformly samples order qty at each step.

Mean return: +$7,223
Fill rate: 93.2%
Stockout: 13.0%

Any learning agent must beat this.

(s, S) RULE-BASED

Industry heuristic

Order up to S=30 when inventory ≤ s=10.

Mean return: +$8,882
Fill rate: 97.1%
Stockout: 9.9%

Real retail systems use exactly this rule.

Q-LEARNING

Learning agent

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max Q(s',a') - Q(s,a)]$$

Mean return: +$8,924
Fill rate: 99.76%
Stockout: 0.62%

Matches revenue. Eliminates stockouts.

Key Results

200 paired episodes (greedy policy, no exploration) · Episode seeds 1000–1199

0.62%
Q-Learning stockout rate

vs 9.9%
(s,S) rule stockout rate

99.76%
Q-Learning fill rate

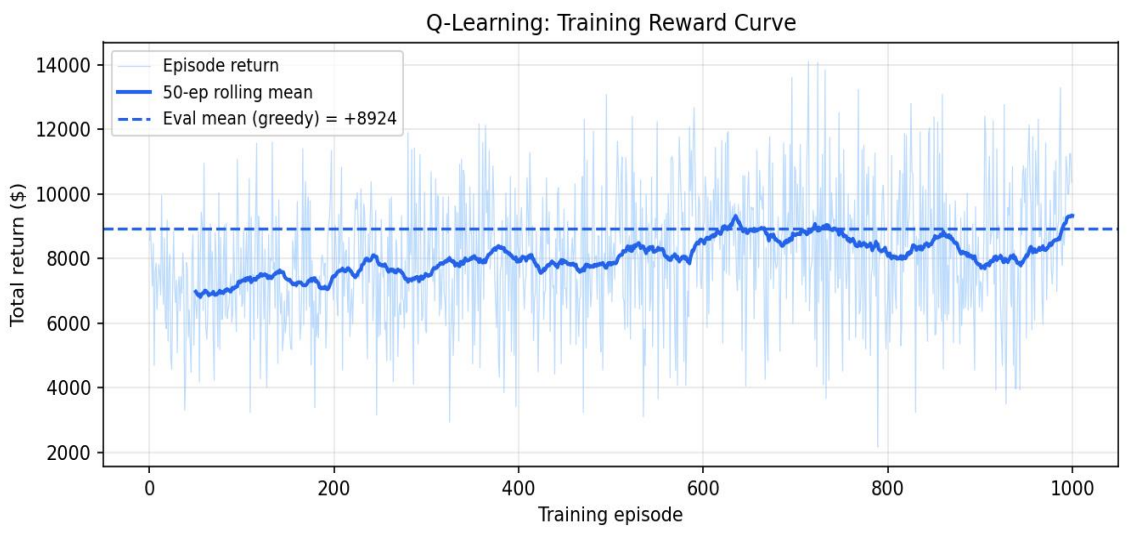
p=0.32
Revenue diff: not significant

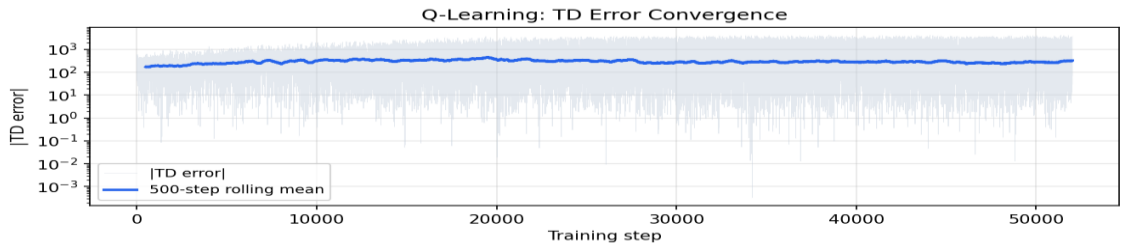
Agent	Mean Return	Std	Fill Rate	Stockout Rate
Q-Learning	**+$8,924**	**$2,120**	**99.76%**	**0.62%**
(s,S) Rule	+$8,882	$1,612	97.11%	9.92%
Random	+$7,223	$1,555	93.15%	12.95%

Revenue difference Q vs (s,S): p=0.32 (not significant). Real advantage: 16× stockout reduction. Revenue metric alone is not sufficient.

Training & Convergence

1,000 episodes · ~1 second on any laptop · no external RL library





+7,265

Mean return
episodes 1–200
(exploring)

+8,453

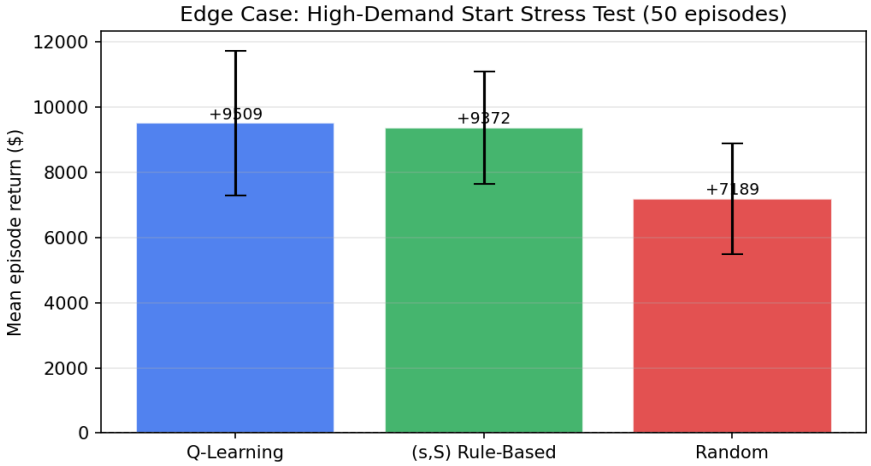
Mean return
episodes 801–1000
(converged)

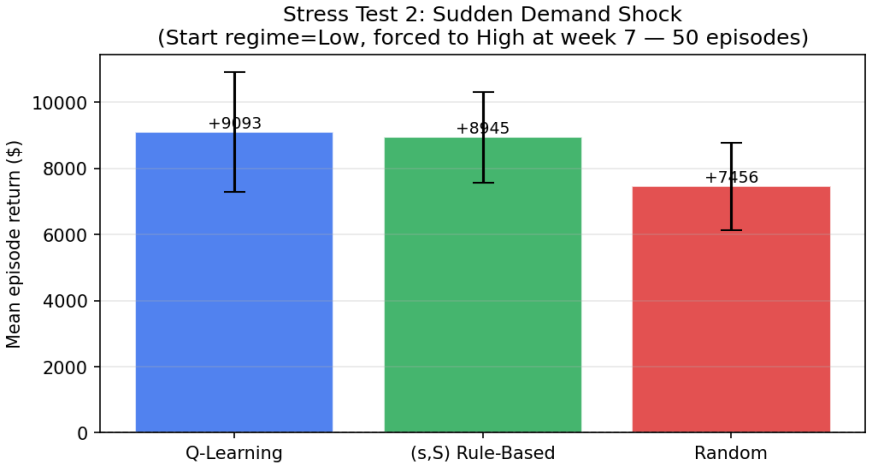
+16%

Improvement
over training

Stress Tests

Two adversarial scenarios - do agents adapt when conditions change?



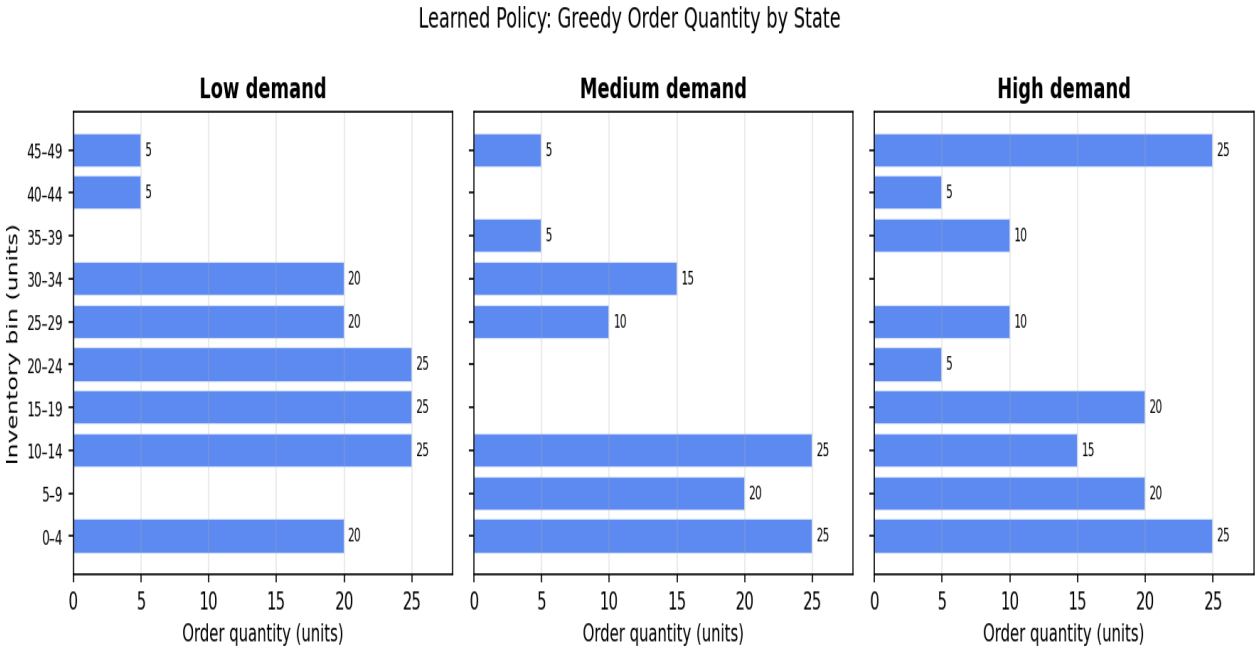


Scenario	Q-Learning	(s,S) Rule	Random	Interpretation
High-demand start	**+$9,509**	+$9,372	+$7,189	Spike start: agent adapts fast
Sudden demand shock	**+$9,093**	+$8,945	+$7,456	Shock mid-ep: regime-blind rule lags

Both tests confirm: Q-learning reads the regime state immediately. The (s,S) rule has no such signal - it reacts only through inventory depletion.

What the Agent Learned

Greedy policy: order quantity for each (inventory bin, demand regime) state



Low stock + high demand → max order (25 units)

High stock + any demand → order little or nothing

Same inventory, different regime → different order

(s,S) rule ignores regime - agent exploits it

Failure Analysis

Five failure modes to monitor before any production deployment

Reward Hacking

Agent aggressively minimises stockouts. If supplier MOQ < 5 units, it may order more frequently and inflate fixed ordering costs in ways the reward doesn't penalise.

Higher Variance

σ=$2,120 vs $1,612 for (s,S). In the worst decile (P10), Q-learning underperforms by ~$608/year. Managers on downside metrics may prefer the rule.

Simulator Gap

Environment uses stationary Poisson demand. Real retail has calendar effects, cross-SKU correlation, promotions. Agent was never trained on these.

Latent State

Demand regime is observed in simulation but latent in production. It must be inferred from recent sales - a separate classification problem not yet solved.

Non-Stationarity

Q-table is static after training. Sustained demand drift (new store format, new competitors) will degrade performance silently and without warning.

Recommendation

SHADOW DEPLOY - observe for one selling season before granting order authority

1. Real-time dashboard comparing agent recommendation to buyer's actual order. Flag divergences > one order tier.
2. Hard cap: agent may never push inventory above physical shelf capacity. Enforce at warehouse level, not just in code.
3. Regime-labelling pipeline. Until a rolling demand classifier is operational, the agent is acting on incomplete state.
4. Six-month re-training cycle on real transaction data, evaluated on held-out weeks before deploying the updated table.

"If this agent learns the wrong policy, who notices, and when?" - Shadow deployment is how we build the answer.